

Java Programming: From C

Chapter: 06. User-Defined Types

Prepared by: Chunghyun Lim (Matriculated in 2019)

Last Updated: May 2026



목차

1. enum

2. class

1. enum

정수·문자열 상수의 문제점

- 정수나 문자열 상수는 값의 의미가 코드에 직접 드러나지 않는다.
- 자료형이 같으면 의미적으로 허용되지 않는 값이라도 컴파일 시점에 이를 검출할 수 없다.
 - 잘못된 값이 런타임까지 그대로 전달될 수 있다.

// 잘못된 예

```
int status = 3;           // 값의 의미를 알기 어렵다.
```

```
String role = "ADMIN"; // 오타나 잘못된 값 사용 가능성
```

// 조금 더 괜찮은 예

```
final int STATUS_SHIPPED = 3;
```

```
final String ROLE_ADMIN = "ADMIN";
```

1. enum

enum이란?

- enum(열거형)은 서로 관련된 상수 집합을 하나의 이름 있는 자료형으로 정의하는 사용자 정의 자료형이다.
- 허용 가능한 값의 범위를 명확하게 제한함으로써 의미가 분명한 상태를 표현하고 타입 안정성을 높인다.
- 정수나 문자열 상수를 직접 사용하는 방식에 비해 의미적으로 잘못된 값의 사용을 컴파일 시점에 예방할 수 있다.

// 열거형명.java에서 정의

```
enum 열거형명 {  
    VALUE1,  
    VALUE2,  
    VALUE3  
}
```

// 사용할 때

```
열거형명 변수명 = 열거형명.VALUE1;
```

// OrderStatus.java에서 정의

```
enum OrderStatus {  
    CREATED,  
    PAID,  
    SHIPPED  
}
```

// 사용할 때

```
OrderStatus status = OrderStatus.SHIPPED;
```

1. enum

enum과 타입 안정성

- enum은 독립적인 자료형이므로 서로 다른 enum 자료형 간의 대입은 허용되지 않는다.

```
// OrderStatus.java
```

```
enum OrderStatus { CREATED, PAID, SHIPPED }
```

```
// PaymentStatus.java
```

```
enum PaymentStatus { READY, COMPLETED, FAILED }
```

```
OrderStatus orderStatus = PaymentStatus.COMPLETED; // 컴파일 에러
```

1. enum

enum과 switch 문

- enum은 switch 문과 함께 사용할 때 허용 가능한 모든 상태를 명확하게 나열할 수 있다.
 - 누락된 상태를 논리적으로 점검하기 쉬워지며 코드의 안정성과 가독성이 향상된다.
- enum을 사용하는 switch 문에서 일반적으로 default는 논리적으로 발생하지 않아야 하는 상태를 감시하기 위한 방어 코드로 작성한다.

```
switch (orderStatus) {  
    case CREATED: // 정상 처리 대상  
    case PAID:     // 정상 처리 대상  
    case SHIPPED: // 정상 처리 대상  
    default:  
        // 논리적으로 도달할 수 없는 상태에 대한 방어 코드(assert 또는 예외)  
}
```

- enum 값이 추가되었으나 switch 문에서 이를 처리하지 않으면 default의 방어 코드를 통해 조기에 식별할 수 있다.

1. enum

enum과 사용 시 주의점

- enum의 값은 정의된 순서를 기준으로 내부적인 순서를 가진다.
 - enum 상수는 선언된 순서에 따라 0부터 시작하는 순서값을 가지며 ordinal()은 그 순서값을 정수로 반환한다.

```
OrderStatus orderStatus = OrderStatus.CREATED;
```

```
orderStatus.name();    // "CREATED"
```

```
orderStatus.ordinal(); // 0
```

- enum의 값(선언 순서)에 의존하는 코드는 가독성과 유지보수성을 저해한다.
- enum은 숫자를 대체하기 위한 수단이 아니라 의미가 명확한 상태와 종류를 표현하기 위한 수단이다.
 - ordinal() 값을 로직이나 저장 용도로 사용하는 것은 지양한다.

1. enum

enum 사용 기준

- 상태, 종류, 역할처럼 선택지가 명확히 제한된 값은 enum으로 표현한다.
- 정수나 문자열 상수만으로 의미를 구분해야 하는 값은 enum으로 대체한다.
- enum 이름은 클래스처럼 PascalCase로 작성한다.
- enum 상수는 모두 대문자로 작성하며 여러 단어로 이루어진 경우 언더스코어(_)로 구분한다.
- 하나의 enum은 열거형 이름과 같은 파일에 정의한다.

2. class

구조체의 문제점

- 구조체(struct)는 서로 관련된 여러 데이터를 묶어 하나의 이름 있는 자료형으로 정의하는 사용자 정의 자료형이다.
- 데이터만 묶는 자료형은 각 데이터의 유효 범위와 데이터들 사이의 제약을 자료형 차원에서 강제하기 어렵다.
- 구조체는 데이터와 동작이 분리된 형태로 사용되기 쉬워 상태 변경을 한곳에서 관리하기 어렵다.

```
typedef struct bank_account {  
    int balance;  
    int is_locked;  
} bank_account_t;
```

```
bank_account_t account;
```

```
account.is_locked = 1;
```

```
account.balance = -999999; // 잠금 여부와 무관하게 상태를 직접 변경할 수 있음
```

2. class

사람이 세상을 인식하는 방식

- 사람은 세상의 대상을 데이터의 묶음으로 이해하지 않는다.
- 사람은 세상의 대상을 상태와 동작을 갖는 하나의 단위(객체)로 인식한다.
 - 객체는 동작을 수행한다.
 - 동작은 객체의 상태를 변경한다.
 - 예: 계좌는 잔액이 있고 입금/출금이라는 동작을 한다.

2. class

class란?

- class는 상태와 동작을 하나로 묶어 하나의 이름 있는 자료형으로 정의하는 사용자 정의 자료형이다.
- class는 객체가 가질 수 있는 상태와 수행할 수 있는 동작을 함께 정의한다.
- class를 사용하면 상태 변경을 동작으로 한정해 객체의 사용 방식을 일관되게 관리하기 쉽다.

// 클래스명.java에서 정의

```
class 클래스명 {  
    자료형 상태이름;  
    반환형 동작이름() { }  
}
```

// 사용할 때

```
클래스명 변수명 = new 클래스명();  
변수명.동작이름();
```

```
class BankAccount {  
    private int balance;  
    private boolean isLocked;  
    void withdraw(int amount) {  
        if (isLocked) { return; }  
        if (amount <= 0 || amount > balance) { return; }  
        balance -= amount;  
    }  
}
```

2. class

클래스와 객체

- class는 객체를 만들기 위한 설계(정의)이다.
- 기본 자료형으로 변수를 선언하듯 class 타입으로 변수를 선언하고 new로 객체를 생성한다.
- 하나의 class로 여러 객체를 생성할 수 있으며 각 객체는 서로 다른 상태를 가진다.

```
class BankAccount {  
    private int balance;  
    private boolean isLocked;  
    void deposit(int amount) { /* ... */ }  
    void withdraw(int amount) { /* ... */ }  
}  
  
BankAccount primaryAccount = new BankAccount();  
BankAccount secondaryAccount = new BankAccount();
```

```
primaryAccount = {BankAccount@747}  
  balance = 0  
  isLocked = false  
secondaryAccount = {BankAccount@748}  
  balance = 0  
  isLocked = false
```

2. class

객체는 참조형이다.

- Java에서 class 타입 변수에는 객체 자체가 아니라 그 객체를 가리키는 참조가 저장된다.
- 같은 객체를 가리키는 변수라면 한 변수를 통해 수행한 동작의 결과로 발생한 상태 변화를 다른 변수에서도 관찰할 수 있다.

```
BankAccount primaryAccount = new BankAccount();
```

```
BankAccount aliasAccount = primaryAccount;
```

```
aliasAccount.deposit(1000);
```

```
primaryAccount = null;
```

```
// primaryAccount.deposit(500);
```

```
primaryAccount = new BankAccount();
```

```
primaryAccount = {BankAccount@747}  
  balance = 0  
  isLocked = false
```

```
primaryAccount = {BankAccount@747}  
  balance = 0  
  isLocked = false
```

```
aliasAccount = {BankAccount@747}  
  balance = 0  
  isLocked = false
```

```
primaryAccount = {BankAccount@747}  
  balance = 1000  
  isLocked = false
```

```
aliasAccount = {BankAccount@747}  
  balance = 1000  
  isLocked = false
```

```
primaryAccount = null  
aliasAccount = {BankAccount@747}  
  balance = 1000  
  isLocked = false
```

```
primaryAccount = {BankAccount@749}  
  balance = 0  
  isLocked = false
```

```
aliasAccount = {BankAccount@747}  
  balance = 1000  
  isLocked = false
```

2. class

멤버 변수와 메서드

- 멤버(member)는 class에 선언되는 구성요소를 말하며 대표적으로 멤버 변수(필드)와 메서드가 있다.
- 멤버 변수/필드(member variable/field)는 class 안에 선언된 변수로 객체의 상태를 표현하는 데이터를 저장한다.
- 메서드(method)는 class 안에 정의된 동작으로 객체가 수행할 수 있는 동작을 정의하며 객체의 상태를 읽거나 변경할 수 있다.

```
class BankAccount {  
    private int balance;           // 멤버 변수: 상태  
    void withdraw(int amount) { /* ... */ } // 메서드: 동작  
    void deposit(int amount) { /* ... */ }  // 메서드: 동작  
}
```

```
BankAccount account = new BankAccount(); // 객체 생성 -> account에 객체 참조 저장  
account.withdraw(1000);                  // 객체의 메서드 호출
```

2. class

멤버 접근: '.' 연산자

- '.' 연산자는 객체의 멤버에 접근할 때 사용한다.
- 접근 가능한 멤버 변수라면 객체.멤버 변수 형태로 상태 값을 읽거나 변경할 수 있다.
- 객체.메서드(...) 형태로 객체가 동작을 수행하도록 요청할 수 있다.

```
BankAccount account = new BankAccount(); // 객체 생성 -> account에 객체 참조 저장
account.withdraw(1000);                    // 객체의 메서드 호출

// account.balance = 1000;                // 멤버 변수 접근(쓰기) 예시 - 현재는 허용되지 않음
// int balance = account.balance;         // 멤버 변수 접근(읽기) 예시 - 현재는 허용되지 않음
```

2. class

멤버 변수의 기본값과 초기화 문제

- 객체를 생성하면 멤버 변수는 자료형에 따라 기본값으로 자동 초기화된다.
- 기본값은 의도한 초기 상태와 다를 수 있다.

구분	자료형	기본값
정수형	byte	0
	short	0
	int	0
	long	0L
실수형	float	0.0f
	double	0.0
문자형	char	'\u0000' (NUL)
논리형	boolean	false
참조형	참조형	null

```
class BankAccount {  
    private int balance;  
    private boolean isLocked;  
    void deposit(int amount) { /* ... */ }  
    void withdraw(int amount) { /* ... */ }  
}  
  
BankAccount account = new BankAccount();  
/* balance = 0, isLocked = false로 자동 초기화 */
```


2. class

생성자

- 생성자(constructor)는 객체가 생성될 때 자동으로 호출되는 특별한 멤버이다.
- 생성자를 통해 객체의 멤버 변수를 의도한 초기 상태로 초기화할 수 있다.
- 생성자는 매개 변수 목록이 서로 다른 형태로 여러 개 정의될 수 있으며 이를 생성자 오버로딩(constructor overloading)이라고 한다.
- 생성자는 반환형을 명시하지 않으며 클래스 이름과 같은 이름으로 정의한다.

```
class 클래스명 {  
    클래스명() { /* ... */ }  
}
```

```
class BankAccount {  
    private int balance;  
    BankAccount() { this.balance = 0; }  
    BankAccount(int balance) { this.balance = balance; }  
}  
  
BankAccount primaryAccount = new BankAccount();  
BankAccount secondaryAccount = new BankAccount(5000);
```

2. class

생성자가 필요한 이유

- 클래스에 생성자를 하나도 정의하지 않으면 컴파일러가 매개 변수가 없는 기본 생성자(default constructor)를 자동으로 제공한다.
- 객체는 생성 시점부터 클래스가 의도한 유효한 상태여야 한다.
- 생성자는 객체 생성에 필요한 입력을 강제하여 초기 조건을 생성 시점에 보장할 수 있다.
- 생성자는 초기화 로직을 한 곳에 모아 중복을 줄이고 초기화 누락과 같은 실수를 줄일 수 있다.

```
class Human {  
    private Sex sex;  
    /* 생성자를 하나라도 정의하면 기본 생성자는 자동으로 제공되지 않음 */  
    Human(Sex sex) { this.sex = sex; }  
}  
  
// Human human = new Human();      // 컴파일 에러  
Human human = new Human(Sex.MALE); // 생성 시점에 초기 조건 보장
```

2. class

접근 제어자

- 접근 제어자(access modifier)는 클래스와 멤버에 대해 어디서 접근할 수 있는지를 규정한다.
- Java의 대표적인 접근 제어자는 다음과 같다.
 - public: 어디서든 접근할 수 있다.
 - private: 선언된 클래스 내부에서만 접근할 수 있다.

```
class BankAccount {  
    private int balance;  
  
    public BankAccount(int balance) { /* ... */ }  
    public void deposit(int amount) { /* ... */ }  
    public void withdraw(int amount) { /* ... */ }  
}  
  
BankAccount account = new BankAccount(5000);  
account.deposit(1000);  
  
// account.balance = 0; // 컴파일 에러
```

2. class

private 멤버 변수의 활용

- private 멤버 변수는 선언된 클래스 내부에서만 직접 접근할 수 있다.
- private 멤버 변수의 상태를 외부에 제공하려면 public 메서드로 읽기 동작을 제공한다.
- private 멤버 변수의 상태를 외부에서 변경해야 한다면 public 메서드를 통해 변경 동작을 제공한다.
 - 메서드 안에서 필요한 검증과 규칙을 강제한다.

```
class BankAccount {  
    private int balance;  
  
    public BankAccount(int balance) { /* ... */ }  
    public int getBalance() { /* ... */ }  
    public void setBalance(int balance) { /* ... */ }  
}
```

```
BankAccount account = new BankAccount(5000);  
account.setBalance(8000);  
  
int currentBalance = account.getBalance();
```

2. class

코딩 컨벤션: getter / setter

- 생성자는 객체가 생성 시점부터 유효한 상태가 되도록 초기 조건을 강제한다.
- 멤버 변수는 `private`로 선언한다.
- getter는 외부 읽기 접근이 필요할 때만 제공한다.
 - getter가 객체의 참조를 노출하는 경우에는 외부 수정 가능성을 신중히 고려한다.
- setter는 원칙적으로 지양한다.
 - 상태 변경은 데이터를 직접 바꾸는 것이 아니라 동작을 수행한 결과로 표현하는 것이 이상적이다.
 - 외부에서 상태 변경 책임을 가져야 하는 경우에만 setter를 제한적으로 제공하고, 메서드 내부에서 유효성 검사와 규칙을 강제한다.

2. class

static 키워드

```
class BankAccount {  
    private int balance;  
    private static int accountCount;  
    public BankAccount(int balance) { /* ... */ }  
    public static int getAccountCount() { /* ... */ }  
}  
  
int totalAccountCount;  
  
BankAccount primaryAccount = new BankAccount(5000);  
totalAccountCount = BankAccount.getAccountCount();  
  
BankAccount secondaryAccount = new BankAccount(8000);  
totalAccountCount = BankAccount.getAccountCount();
```

- static 멤버는 객체가 아니라 클래스에 속한다.
- static 멤버는 객체를 만들지 않아도 클래스이름.멤버로 접근한다.
- 해당 클래스의 모든 객체는 같은 static 멤버 값을 공유한다.

```
10 01 totalAccountCount = 0  
-----  
10 01 totalAccountCount = 1  
☰ primaryAccount = {BankAccount@748}  
Ⓜ balance = 5000  
-----  
10 01 totalAccountCount = 2  
☰ primaryAccount = {BankAccount@748}  
Ⓜ balance = 5000  
☰ secondaryAccount = {BankAccount@749}  
Ⓜ balance = 8000
```

Thank you

Wishing you an enjoyable and meaningful learning experience.
May you grow into an engineer who learns with joy and shares with others.

For inquiries, contact: potterLim0808@gmail.com

